DOCENTE	Paolo BELLONI [paolo.belloni@its-ictpiemonte.it]
Unità Formativa	**Sistemi di Virtualizzazione**
Argomento	Macchina Virtuale e Openstack

Intervento realizzato da

Infrastruttura Virtuale

Componenti software (softwarizzate, astratte) che abilitano l'esecuzione di applicazioni senza disporre di una macchina fisica

- **Macchine Virtuali**
- **Container**
- **K8S**
- **Cloud Service Provider**

Openstack

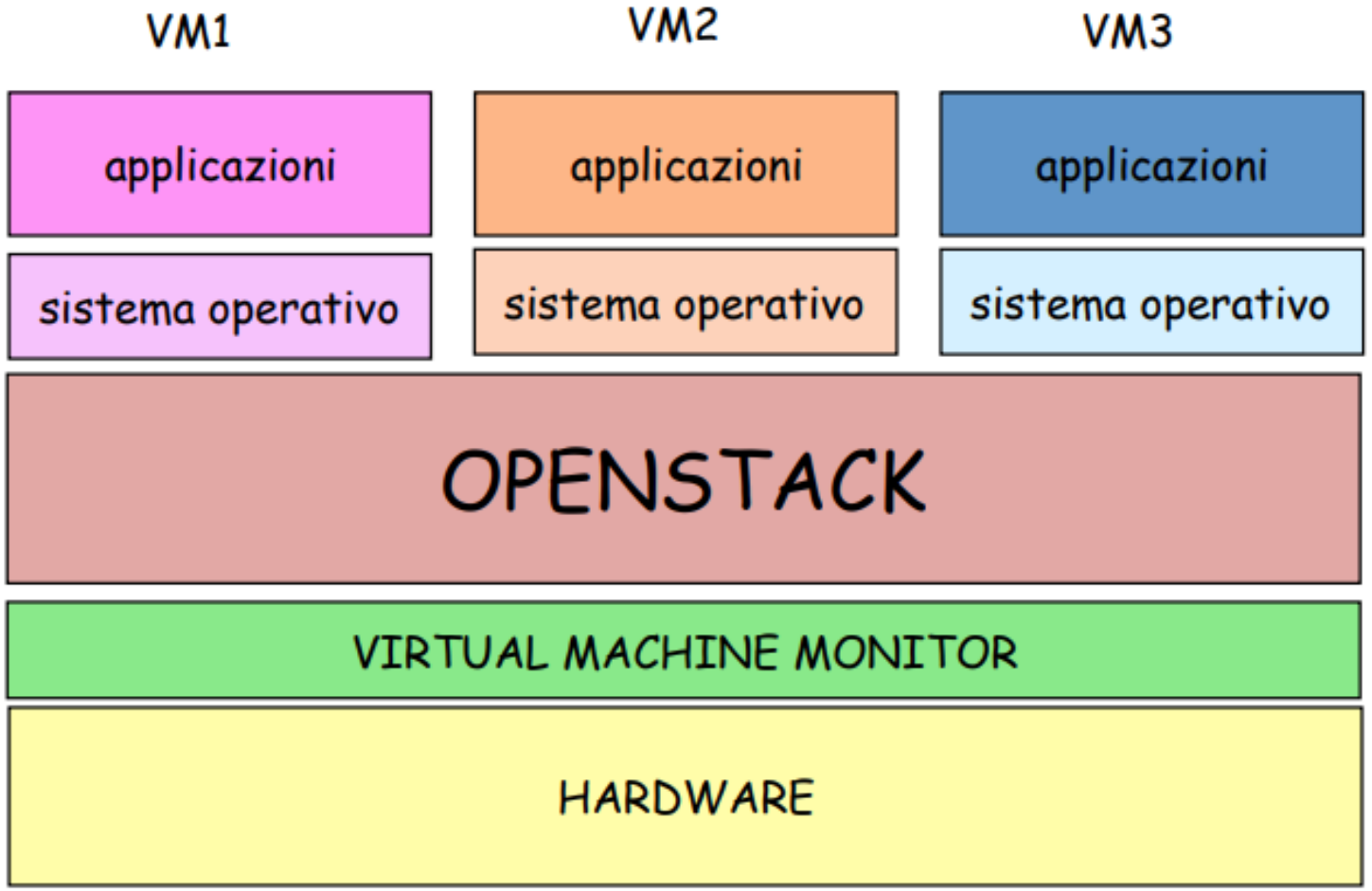
Come si realizza il Cloud Computing?

- Attraverso software non troppo diverso da hypervisor, ma creato per gestire il cloud.
- Orchestratore di macchina virtuali.

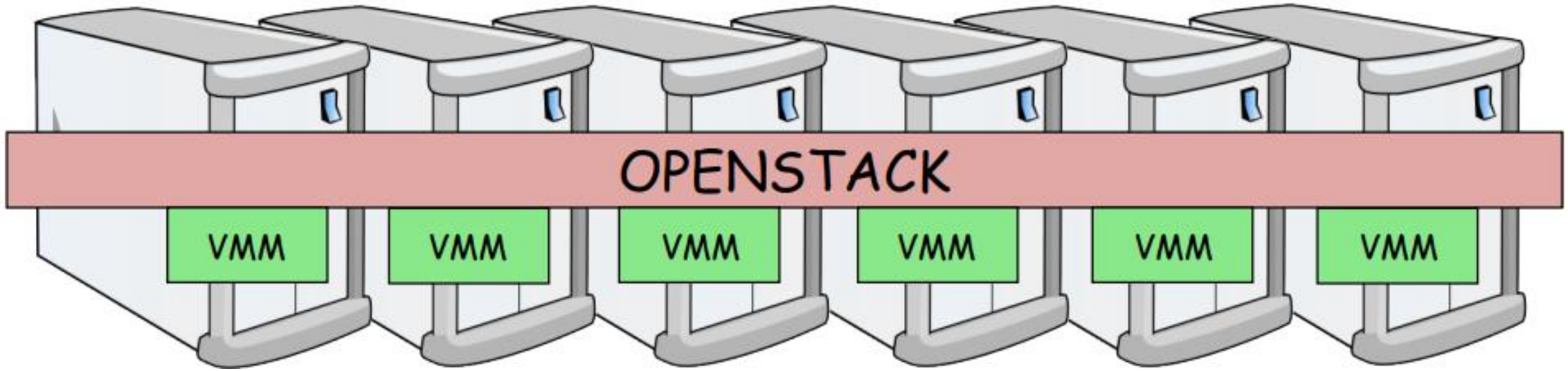
- **Openstack**: piattaforma software per il management di un cloud.
- Progetto iniziato nel 2010 dalla collaborazione tra Rackspace e la NASA.
- Open source.

Come si realizza il Cloud Computing? (1)

- Viene installato sul VMM.
- Si avvale delle sue funzionalità per creare un cloud di macchine fisiche

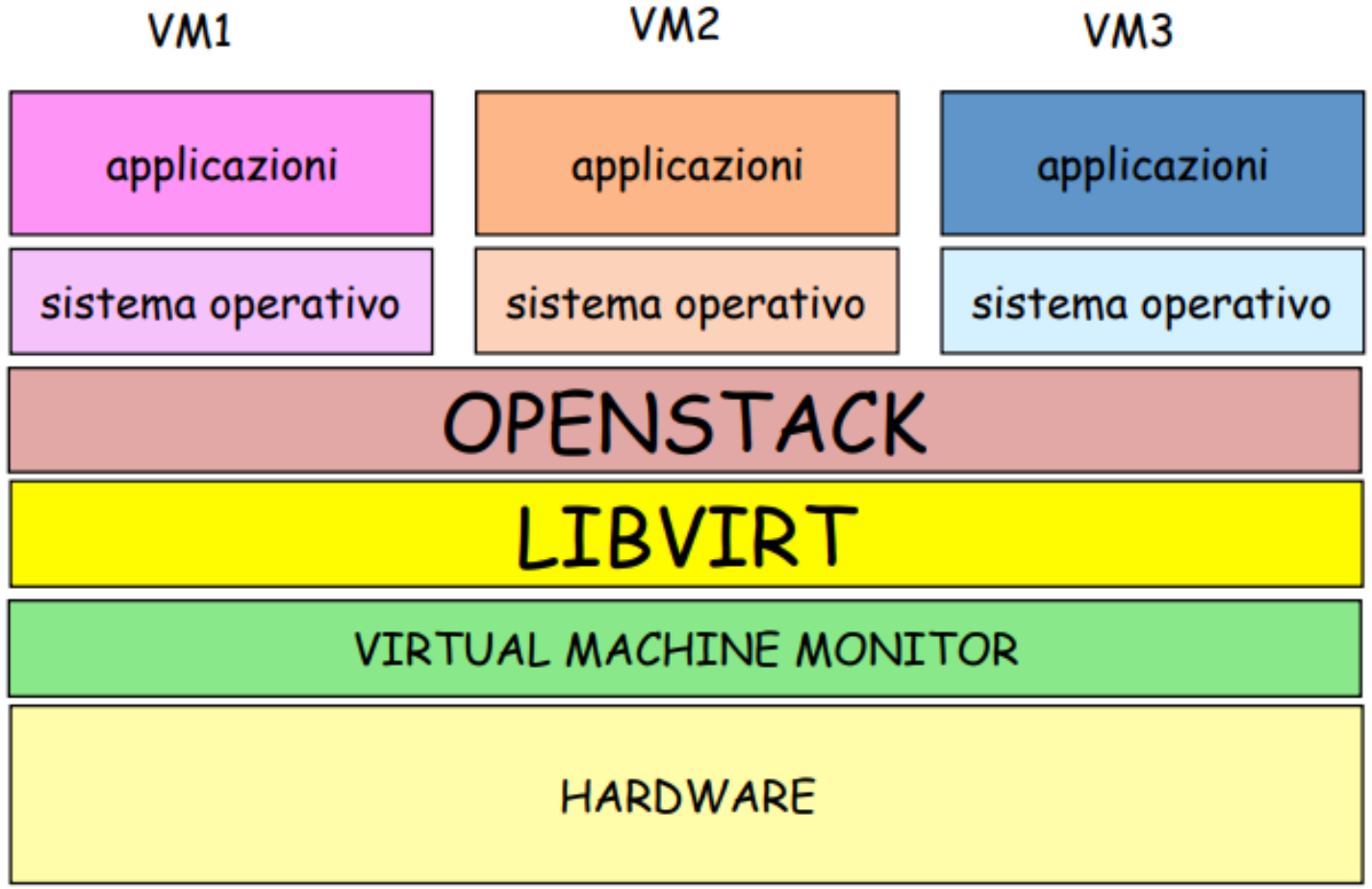


Come si realizza il Cloud Computing? (2)



Libvirt per gestire eterogeneità

- Si avvale di **libvirt**:
- Layer per VMs management (creare, avviare, fermare, modificare, monitorare, migrare).
- Permette di interagire con diversi VMM:
  - ✓Kvm/Qemu,
  - ✓Xen,
  - ✓Vmware ESX,
  - ✓MS Hyper-V
  - ✓…



Gestione delle Risorse in Pool

La Gestione delle Risorse (Resource Management)

Resource Management

Un processo per allocare, a favore delle macchine virtuali (VM), le risorse di una macchina fisica o di un cluster di macchine fisiche, al fine di ottimizzare l'utilizzo complessivo delle risorse.

- A cosa serve la gestione delle risorse?
  - Controlla l'uso delle risorse.
  - Impedisce che una singola VM monopolizzi le risorse.
  - Alloca le risorse opportunamente, in base alle priorità assegnate alle varie VM.
- Le risorse devono essere messe in pool (**Resource Pooling**), allo scopo di gestirle "centralmente".
- Risorse = CPU, memoria, rete e storage.

Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

Il Resource Pooling

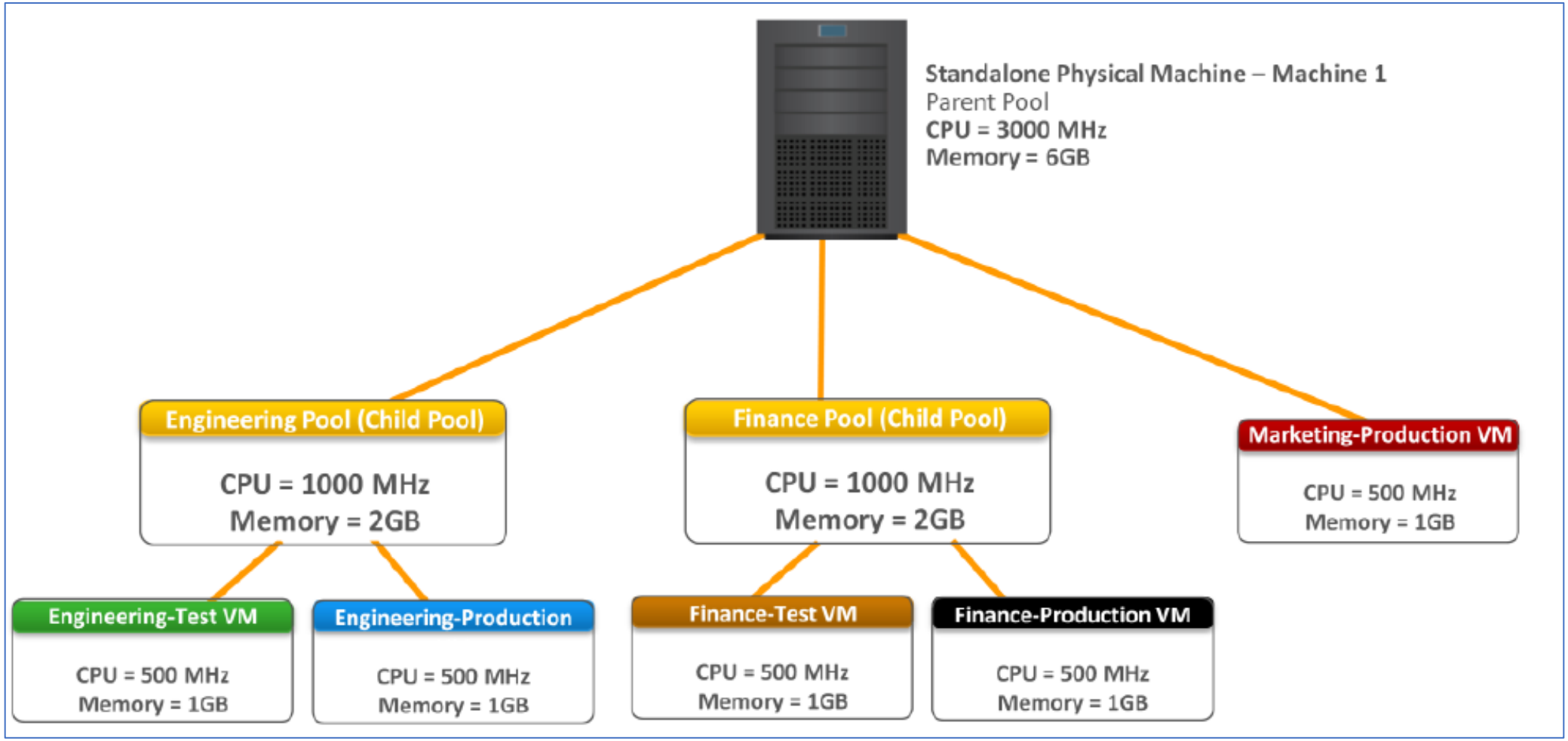
Resource Pooling

È una astrazione logica delle risorse fisiche aggregate, che ne abilita la gestione centralizzata.

- Un pool è creato a partire da una macchina fisica o da un insieme di machine fisiche (cluster).
  - Questo resource pool di partenza è definito **parent resource pool**.
- A partire dal parent resource pool (iniziale), gli Amministratori di sistema possono allocare le risorse ad una applicazione o a delle VM.
  - Le risorse allocate, a partire dal parent pool, per una specifica applicazione o alla singola VM si definiscono **child resource pool**.
- Tre indicatori vengono usati per controllare le risorse consumate dai diversi pool o dalle VM.
  - Si parla di **Key Performance Indicator (KPI).** Questi sono:
  - Reservation;
  - Limit;
  - Share.

Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

Il Resource Pooling: un esempio



Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

Share, Limit e Reservation

- Share Limit e Reservation sono dei parametri (variabili) che controllano le risorse allocate ed utilizzate da un child resource pool o da una VM.

❖Share
- Quantità di CPU o memoria che una VM o un child pool può avere, rispetto al totale delle risorse (di CPU o memoria) del parent pool.
- Individua la priorità di una VM o child pool.

❖Limit
- Massima quantità di CPU o memoria che una VM o un child pool può consumare.
- Configurato alla creazione della VM, si può modificare solo a VM spenta.

❖Reservation
- Quantità di CPU o memoria riservata per una VM o per un child pool.
- Una VM non viene attivata, se la quanità di risorse definite dalla reservation non sono disponibili.
- Se la VM non utilizza tutte le risorse riservate, quelle libere vengono allocate ad altre VM.

Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

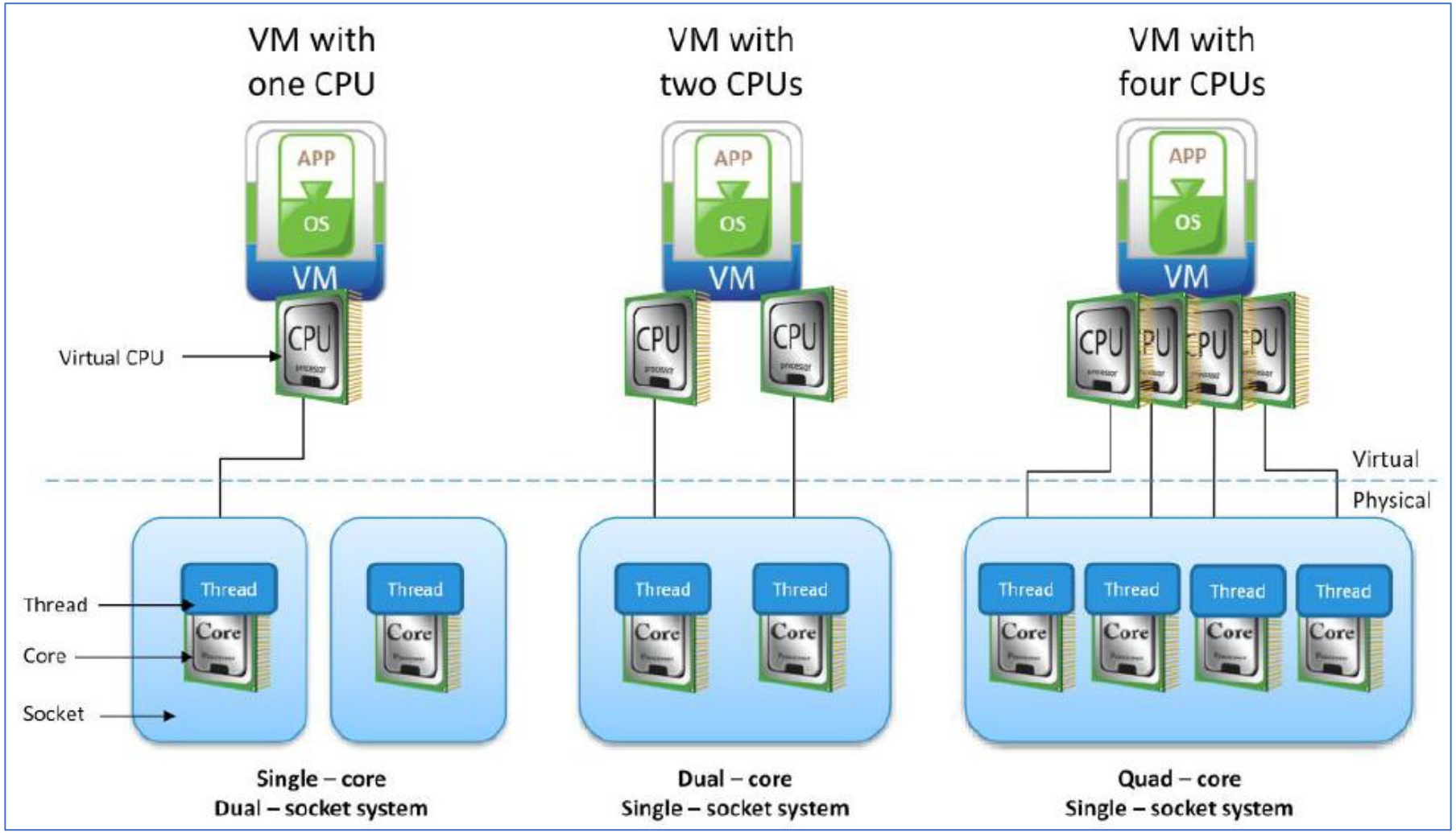
L'ottimizzazione delle risorse di CPU

- Allocare le risorse di una CPU in modo efficiente e appropriato è fondamentale.
- Le moderne CPU sono equipaggiate con core multipli e con hyper-threading.
  - I processori multi-core hanno, in una singola CPU (circuito integrato o socket), diverse unità di processamento (processing unit, a.k.a. core).
  - L'hyper-threading fa apparire una CPU fisica come due o più CPU logiche.
- L'hypervisor definisce e alloca le CPU virtuali sulla CPU fisiche (analogamente a come un SO assegna ai diversi processi – o thread – il tempo di CPU).
- L'hypervisor supporta le funzionalità di multi-core, hyper-threading e CPU load-balancing (bilanciamento del carico) per ottimizzare le risorse della CPU.

Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

CPU: pCPU, L-CPU, vCPU

I processori Multi-Core

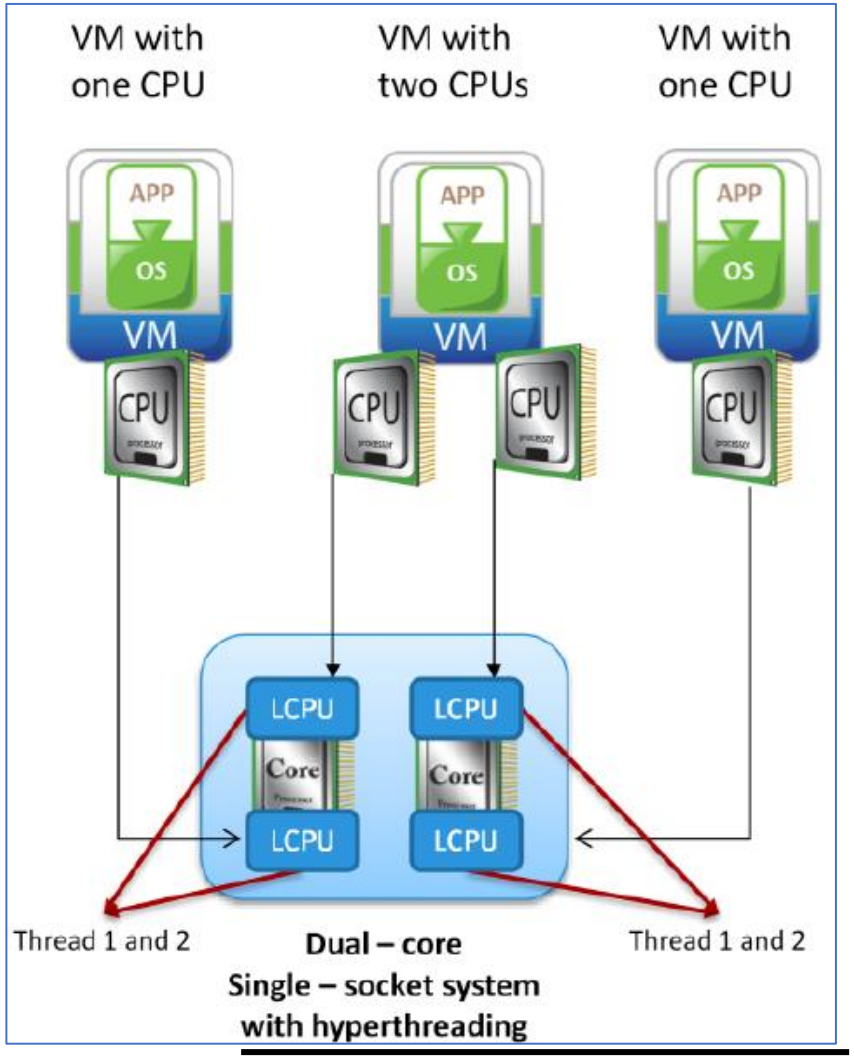


Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

L'hyper-threading

Un **thread** o thread di esecuzione, in informatica, è una suddivisione di un processo in due o più *filoni* (istanze) o sottoprocessi, che vengono eseguiti simultaneamente da un sistema di elaborazione monoprocessore (mono-threading) o multiprocessore (multi-threading) o multicore.

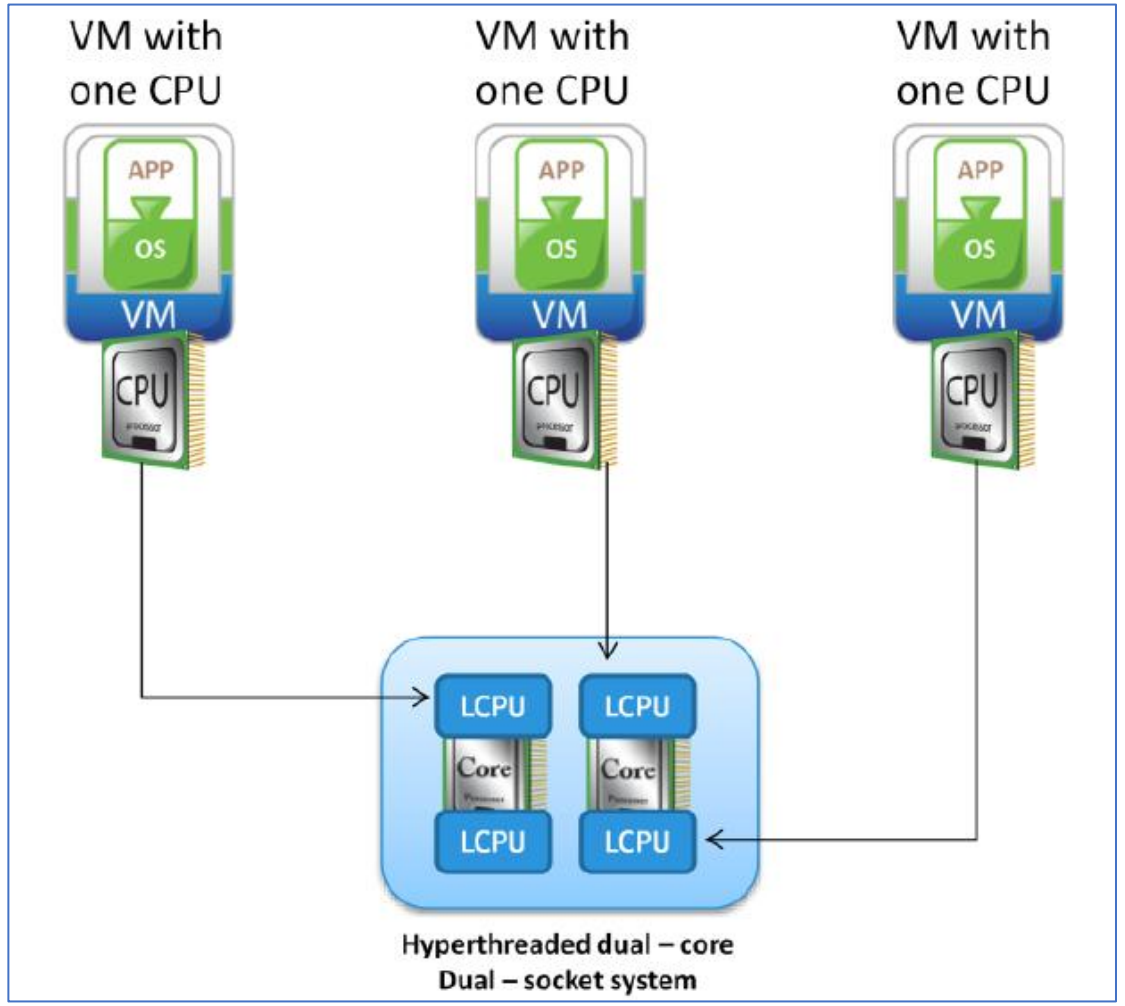
- L'hyper-threading fa apparire una CPU come due CPU logiche (Logical CPU LCPU).
  - In pratica, l'hyper-threading abilita il SO a eseguire due o piu thread contemporaneamente.
- Due LCPU condividono le stesse risorse fisiche.
  - Due thread non possono essere eseguiti contemporaneamente, ma mentre un thread è bloccato, la CPU può eseguire una altro thread.
- L'hypervisor in esecuzione su una CPU che supporta hyper-threading ha prestazioni migliori ed un utilizzo delle risorse più efficiente.



Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

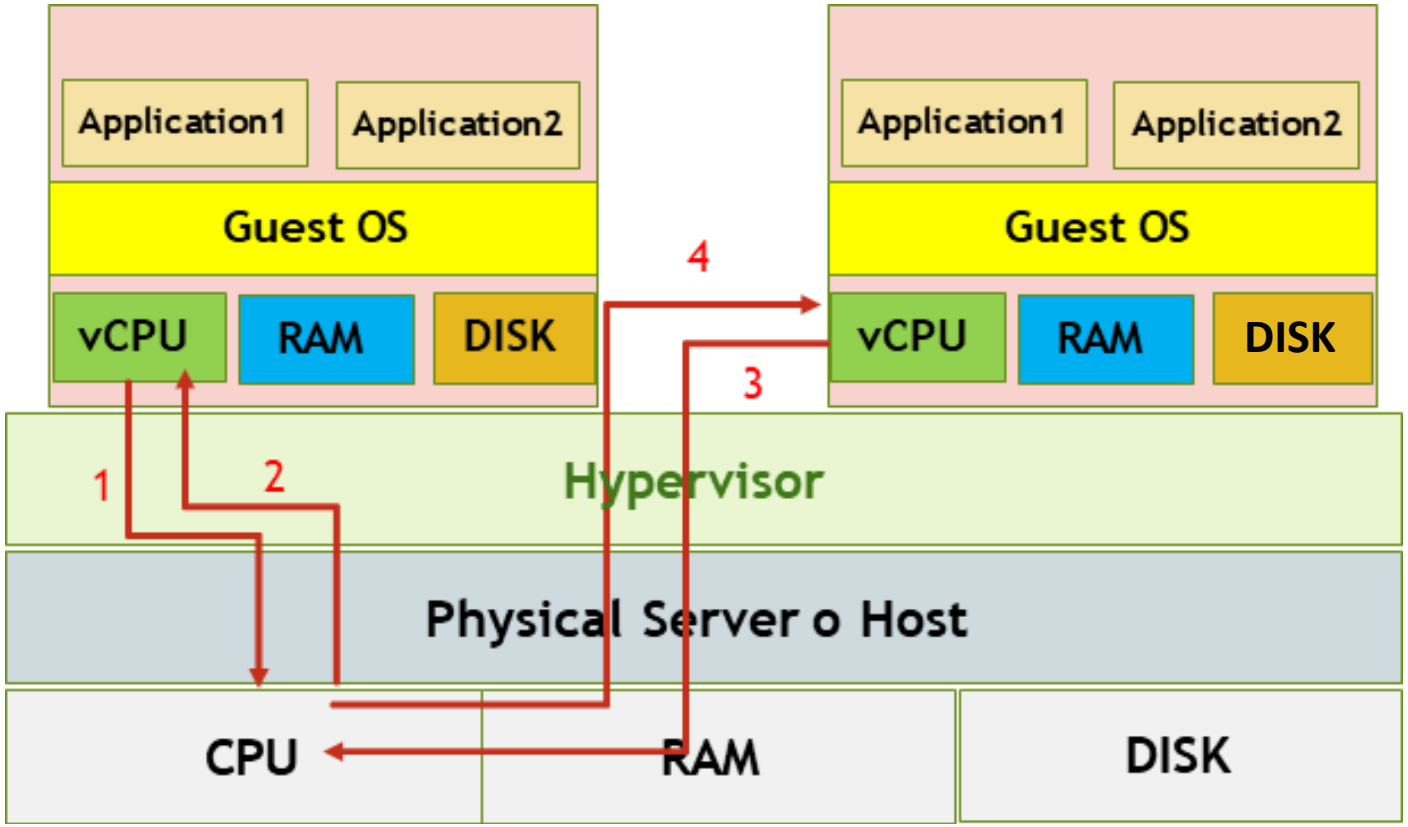
Il bilanciamento di carico della CPU

- Un aspetto fondamentale della gestione delle risorse è il bilanciamento del carico (load balancing) tra le risorse di processamento disponibili nella particolare configurazione considerata.
- Diventa chiave nel caso di sistema multi-processore e in caso l'hyper-threading sia attivo.
  - Un thread viene migrato da una L-CPU troppo utilizzata verso una meno carica.
  - L'hypervisor controlla ad intervalli regolari il carico delle varie L-CPU, al fine di mantenerlo bilanciato.
  - Se una L-CPU non è utilizzata, viene stoppata. Questo permette di liberare le risorse fisiche della CPU e permette alla VM in esecuzione sulla L-CPU dello stesso core di avere più "banda" disponibile.



Source: **EMC²**, Cloud Infrastructure and Service, Student Guide, 2011

Gestione della CPU nella VM



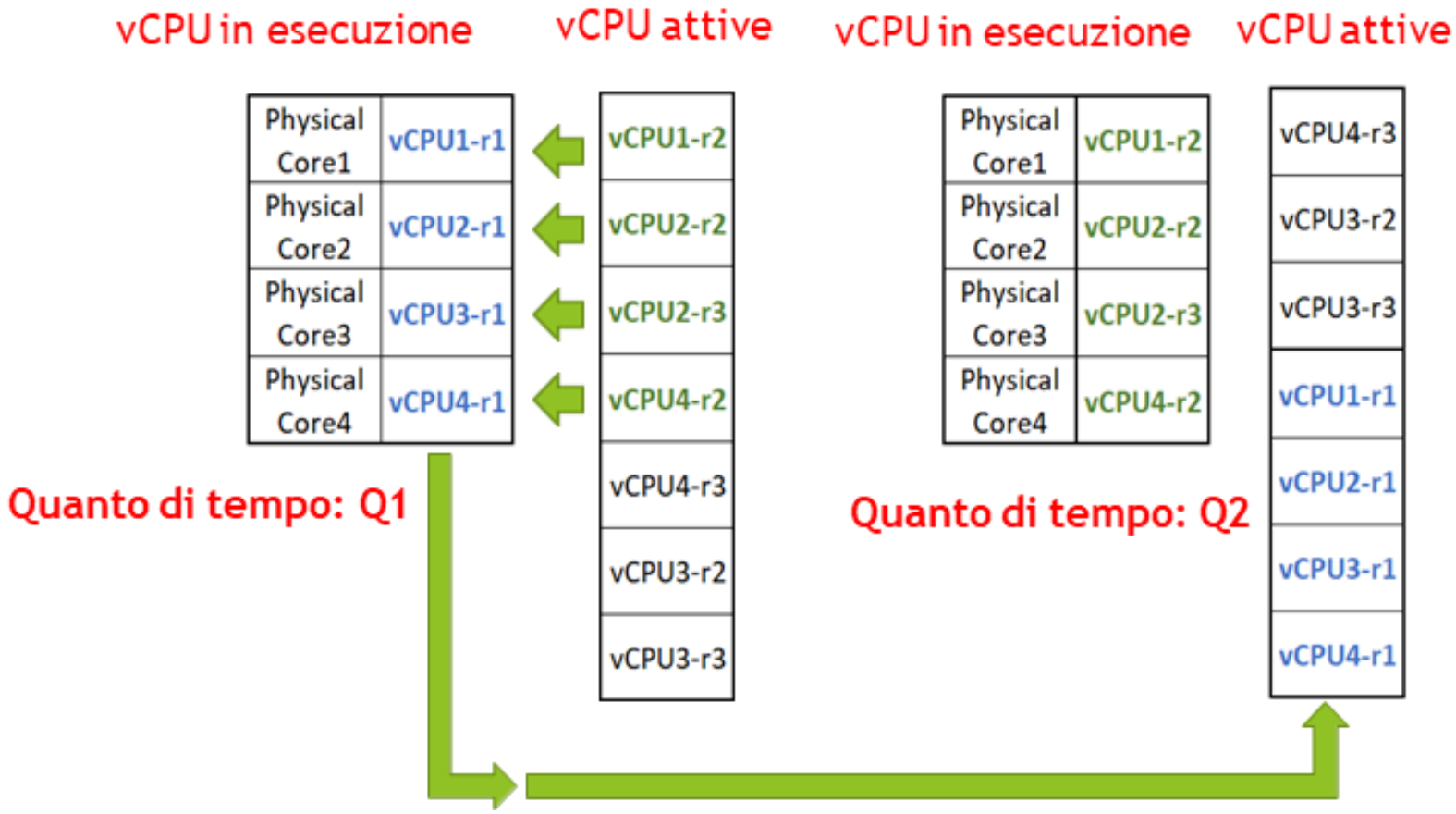
- L'hypervisor assegna gli intervalli di tempo sui processori disponibili nel server host fisico per l'esecuzione delle istruzioni delle CPU virtuali, che indicheremo con vCPU.
- La prima vCPU deve eseguire una serie di comandi sull'hardware virtuale e tale richiesta viene indirizzata all'hypervisor.
- L'hypervisor schedula una sessione di esecuzione per la richiesta di quella vCPU **(1)**.
- La CPU fisica esegue le istruzioni, restituisce i risultati all'hypervisor, che li restituisce alla prima macchina virtuale **(2)**.
- Quindi l'hypervisor assegna l'esecuzione della successiva serie di istruzioni dalla seconda macchina virtuale **(3)** e **(4)**.
- Inoltre, le CPU virtuali non sono associate a CPU fisiche.
- Un hypervisor pianificherà il lavoro per conto di una macchina virtuale su qualsiasi CPU fisica disponibile.

Gestione della CPU nella VM

Numero di CPU	Single Core	Dual Core	Quad Core
1	1	2	4
2	2	4	8
4	4	8	16
8	8	16	32

- La tabella mostra il numero di core disponibili in funzione delle configurazioni del processore e del numero di processori.
- Ogni vendor di sistemi di virtualizzazione dichiara il rapporto max vCPU/CPU.
- Ogni guest utilizza una parte di una CPU fisica e quindi si può allocare più di una vCPU per ogni CPU fisica del server.
- Ogni fornitore ha le proprie indicazioni sul massimo numero di vCPU che possono essere allocate su ogni singolo host.

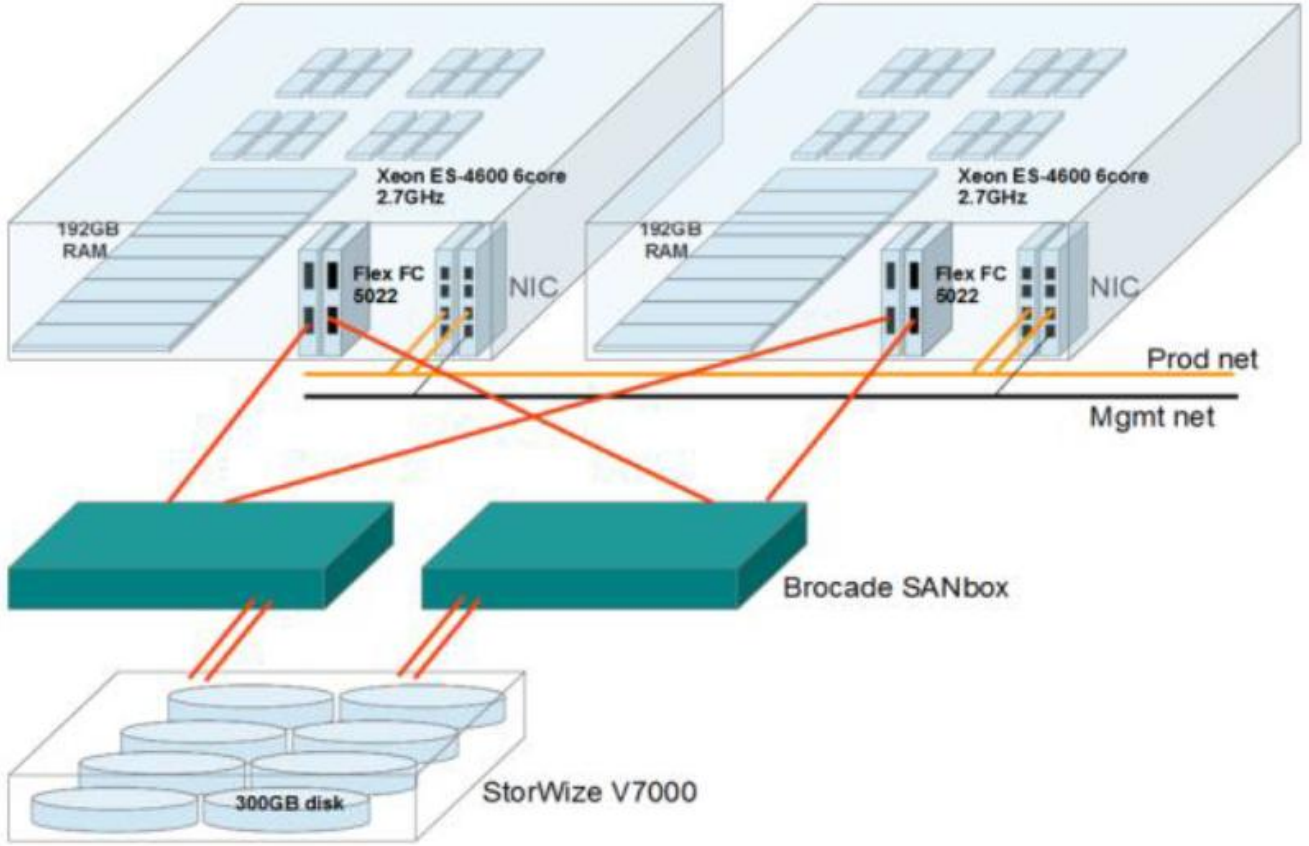
Gestione Quanti di Tempo vCPU



- L'hypervisor gestisce le risorse della macchina fisica esattamente come un sistema operativo.
- La differenza è che le richieste arrivano all'hypervisor da un sistema operativo installato su di una vCPU.

VM – Esempio

Esempio | La Macchina Host o Server Fisico



- Ciascuno dei due server **IBM Flex x440** è equipaggiato come segue:
  - 24 core x86 a 2,7 GHz;
  - 192 GB di RAM;
  - 1 Fiber Channel HBA (Host Bus Adapter) 16 Gbps;
  - 8 porte Ethernet da 1 Gbps.
- La SAN si basa su SAN Brocade e su un'unità IBM StorWize V7000 con 20 dischi SAS da 300 GB e 4 SSD da 250 GB.
- Flex FC 5022 è adattatore SAN montato sul server.

Model	cores	RAM (GB)	HBA	NIC	Description
IBM Flex X440	4xIntel Xeon ES-4600 6core@2.7GHz	192	2 x Flex FC 5022 16Gbps	8 x 1Gbps	Server #1
IBM Flex X440	4xIntel Xeon ES-4600 6core@2.7GHz	192	2 x Flex FC 5022 16Gbps	8 x 1Gbps	Server #2

Esempio | I Server Virtuali

Name	Cores/ RAM	disks	Virtual Server	VM Purpose	VM software
SERVSRV092	2cores /16GB	500GB shared	1.2.6.21	Production	Oracle RDBMS 11gr2 RAC cluster software
SERVSRV093	2cores /16GB	500GB shared	1.2.6.22	Production	Oracle RDBMS 11gr2 RAC cluster software
SERVSRV094	2cores /16GB	60GB + shared filesystem	1.2.6.21	Production	CCApp GW cluster software CCApp AS
SERVSRV095	2cores /16GB	60GB + shared filesystem	1.2.6.22	Production	CCApp GW cluster software CCApp AS
SERVSRV096	2cores /8GB	60GB + shared filesystem	1.2.6.21	Production	CCApp AS
SERVSRV097	2cores /8GB	60GB + shared filesystem	1.2.6.22	Production	CCApp AS
SERVSRV098	1core/2GB	10GB	1.2.6.21	Production	CCApp WS
SERVSRV099	1core/2GB	10GB	1.2.6.22	Production	CCApp WS
SERVSRV100	2cores /16GB	500GB	1.2.6.21	Test/Perf	Oracle RDBMS 11gr2 stand alone
SERVSRV101	2cores /16GB	500GB	1.2.6.22	Acceptance/Dev	Oracle RDBMS 11gr2 Acceptance + Development
SERVSRV102	2cores /16GB	60GB	1.2.6.21	Test/Perf	CCApp GW cluster software CCApp AS
SERVSRV103	2cores /16GB	60GB	1.2.6.22	Test/Perf	CCApp GW cluster software CCApp AS
SERVSRV104	2cores /8GB	60GB	1.2.6.21	Test/Perf	CCApp AS
SERVSRV105	2cores /8GB	60GB	1.2.6.22	Test/Perf	CCApp AS
SERVSRV106	1core/2GB	10GB	1.2.6.21	Test/Perf	CCApp WS
SERVSRV107	1core/2GB	10GB	1.2.6.22	Test/Perf	CCApp WS
SERVSRV109	2cores /16GB	100GB	1.2.6.21	Acceptance/Dev	CCApp AS
SERVSRV110	2cores /16GB	100GB	1.2.6.21	Acceptance/Dev	CCApp AS
SERVSRV111	1core /2GB	10GB	1.2.6.21	Acceptance/Dev	CCApp WS

- Le macchine virtuali costituiscono tre ambienti:
  - Produzione
  - Test/Perf
  - Development/Acceptance
- La potenza richiesta per le macchine (virtuali) è stata stimata in un esercizio di sizing.
- La capacità complessiva del server fisico è superiore alla somma delle macchine virtuali configurate (33 core/cpu).
- È possibile osservare che la potenza delle macchine di Acceptance/Dev e Test/Perf è simile o uguale a quella dell'ambiente di Produzione.
- Questo perché la virtualizzazione consente di presentare al richiedente, cioè la VM, la potenza necessaria quando questa è necessaria.
- Nel caso di un ambiente fisico tutta la potenza dovrebbe essere stata resa SEMPRE disponibile.

VM – Dimensionamento

vCPU | Esempio

Quante vCPU sono disponibili su CPU *Intel® Xeon® Gold 6148*, 2.4 GHz, 2 processors 20 core?

- Numero core totali = 2 x 20 = 40 CPU fisiche (pCPU).
- Senza hyperthreading → 40 vCPU disponibili (1vCPU = 1 thread HW).
- Con hyperthreading → 40 x 2 vCPU = 80 vCPU disponibili (1 vCPU = 1 LCPU).
- Il VMM può praticare l'*overcommit di CPU*, ovvero assegna complessivamente più vCPU ai guest di quante pCPU fisiche siano disponibili, confidando nel fatto che non tutte le vCPU richiedano tempo di esecuzione simultaneamente.
  - In questo modo, le vCPU disponibili possono essere comunemente 80-160 senza degrado prestazionale evidente.
- Si tenga presente che anche hypervisor usa risorse della macchina host, in questo caso utilizza pCPU.
  - Storicamente, si stimava un impatto sulla pCPU pari al 20% della capacita totale ma negli hypervisor recenti si stima un impatto di hypervisor pari al 5% - 10% di pCPU (10% nel caso di VM-exit molto frequenti).
  - Nel nostro caso, le pCPU disponibili diventano 40 x 0.95 = 38 pCPU (che diventano 36 pCPU se overhead di hypervisor stiamato pari al 10%).

vRAM | Esempio 1/3

Quanta vRAM può essere complessivamente assegnata a machine virtuali da VMM su CPU *Intel® Xeon® Gold 6148*, 2.4 GHz, 2 processors 20 core, ipotizzando che su ciascun blade server siano installati 256 GB di RAM fisica?

- pRAM = 256 GB.
- Il VMM può praticare l'*overcommit di RAM*, ovvero assegna complessivamente più vRAM ai guest di quanta pRAM sia disponibile, confidando nel fatto che non tutte le VM richiedano simultaneamente la quantita di RAM definita nel paramento *reservation*.
  - In questo modo, la quantità di vRAM disponibile può essere comunemente 1.2 – 1.5 volte superiore alla pRAM installata, senza degrado prestazionale evidente.
  - NOTA: mentre nel caso di CPU overcommit il degrado prestazionale e' graduale, quando si pratica RAM overcommit oltre il 1.5 questo innesca memory swap frequenti, che comportano un degrado netto ed immediato delle prestazioni.

vRAM | Esempio 2/3

Quanta vRAM può essere complessivamente assegnata a machine virtuali da VMM su CPU *Intel® Xeon® Gold 6148*, 2.4 GHz, 2 processors 20 core, ipotizzando che su ciascun blade server siano installati 256 GB di RAM fisica?

- pRAM = 256 GB.
- Come per CPU, hypervisor occupa RAM, in quanto:
  - programma in esecuzione e quindi residente in RAM (architettura di von Neumann);
  - con strutture dati di controllo per gestire e controllare le VM;
    - ✓ stato di VM,
    - ✓ stato di vCPU (registri e scheduling), p.es. VMCS/VMCB,
    - ✓ configurazione delle VM;
  - page table/EPT/NPT;
  - buffer interni, cahce, networking.
  - Storicamente, si stimava un impatto sulla pRAM pari al 20% della capacità totale ma negli hypervisor recenti si stima un impatto di hypervisor pari al 5% - 10% (tale overhead è sostanzialmente indipendente dalla frequenza delle VM-exit, che incidono invece sul consumo di CPU).

vRAM | Esempio 3/3

Quanta vRAM può essere complessivamente assegnata a machine virtuali da VMM su CPU *Intel® Xeon® Gold 6148*, 2.4 GHz, 2 processors 20 core, ipotizzando che su ciascun blade server siano installati 256 GB di RAM fisica?

- pRAM = 256 GB.

Soluzione.

1. Hypervisor overhead → pRAM effettivamente disponibile = 256 GB x 0.95 = 230 GB
2. RAM overcommit "prudente" → pRAM * 1.3 = 300 GB